

PRACTICAL-10

Task-1: Investigate inode structures using commands such as ls -li, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

```
ruhi0404@Ruhitha:~$ cd ~/Operating_Systems/
ruhi0404@Ruhitha:~/Operating_Systems$ mkdir -p prac10task1
ruhi0404@Ruhitha:~/Operating_Systems$ cd prac10task1
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ pwd
/home/ruhi0404/Operating_Systems/prac10task1
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ nano original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ cat original.txt
This file is used to demonstrate Linux inode structures.

ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -li original.txt
56363 original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ stat original.txt
  File: original.txt
  Size: 58          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 56363       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ruhi0404)   Gid: ( 1000/ruhi0404)
Access: 2026-09-28 14:32:15.339032404 +0000
Modify: 2026-09-28 14:32:06.550484016 +0000
Change: 2026-09-28 14:32:06.550484016 +0000
 Birth: 2026-09-28 14:32:06.546583362 +0000
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ find . -name "original.txt"
./original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ./original.txt
-bash: ./original.txt: Permission denied
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -li original.txt
56363 original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ find . -inum 123456
```

```

Access: (0644/-rw-r--r--)  Uid: ( 1000/ruhi0404)   Gid: ( 1000/ruhi0404)
Access: 2026-09-28 14:32:15.339032404 +0000
Modify: 2026-09-28 14:32:06.550484016 +0000
Change: 2026-09-28 14:32:06.550484016 +0000
Birth: 2026-09-28 14:32:06.546583362 +0000
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ find . -name "original.txt"
./original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ./original.txt
-bash: ./original.txt: Permission denied
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -i original.txt
56363 original.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ find . -inum 123456
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ln -s original.txt symlink
.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -li
total 4
56363 -rw-r--r-- 1 ruhi0404 ruhi0404 58 Sep 28 14:32 original.txt
56360 lrwxrwxrwx 1 ruhi0404 ruhi0404 12 Sep 28 14:33 symlink.txt -> original
.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -li original.txt hardli
nk.txt symlink.txt
ls: cannot access 'hardlink.txt': No such file or directory
56363 -rw-r--r-- 1 ruhi0404 ruhi0404 58 Sep 28 14:32 original.txt
56360 lrwxrwxrwx 1 ruhi0404 ruhi0404 12 Sep 28 14:33 symlink.txt -> original
.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 56360         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/ruhi0404)   Gid: ( 1000/ruhi0404)
Access: 2026-09-28 14:33:51.955424345 +0000
Modify: 2026-09-28 14:33:44.342635438 +0000
Change: 2026-09-28 14:33:44.342635438 +0000
Birth: 2026-09-28 14:33:44.342635438 +0000
ruhi0404@Ruhitha:~/Operating_Systems/prac10task1$ ls -li
total 4
56363 -rw-r--r-- 1 ruhi0404 ruhi0404 58 Sep 28 14:32 original.txt
56360 lrwxrwxrwx 1 ruhi0404 ruhi0404 12 Sep 28 14:33 symlink.txt -> original
.txt

```

Observation:

The inode structure of files was successfully investigated using the `ls -i`, `stat`, and `find` commands. A hard link and a symbolic link were created for the original file, and their inode numbers and link counts were compared. The hard link shared the same inode number as the original file, showing that both directory entries referred to the same underlying file data. In contrast, the symbolic link had a separate inode and stored the pathname of the target file. Modifying the hard link also modified the original file because they shared the same inode. After deleting the original filename, the hard link remained accessible while the symbolic link became dangling because its target pathname no longer existed. Thus, the practical successfully demonstrated inode allocation, hard links, symbolic links, and their effects on file access and inode structures.

Task-2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity.

```
GNU nano 7.2                                input.txt *
Memory-mapped I/O allows a file to be mapped directly into
the process address space. This practical compares mmap()
with traditional read() and write() operations.
```

```
GNU nano 7.2                                practical2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <sys/time.h>

#define BUFFER_SIZE 4096

double get_time()
{
    struct timeval tv;

    gettimeofday(&tv, NULL);

    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

/*
 * Traditional file I/O using read() and write()
 */
void traditional_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    src_fd = open(source, O_RDONLY);

    if (src_fd == -1)
    {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }
}
```

```

        perror("write failed");
        close(src_fd);
        close(dest_fd);
        exit(EXIT_FAILURE);
    }
}

if (bytes_read == -1)
{
    perror("read failed");
}

close(src_fd);
close(dest_fd);
}

/*
 * Memory-mapped file I/O using mmap()
 */
void mmap_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    struct stat file_info;
    size_t file_size;

    char *src_map;
    char *dest_map;

    src_fd = open(source, O_RDONLY);

    if (src_fd == -1)
    {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    if (fstat(src_fd, &file_info) == -1)

```

```
    close(src_fd);
    close(dest_fd);
    exit(EXIT_FAILURE);
}

if (file_size == 0)
{
    close(src_fd);
    close(dest_fd);
    return;
}

/*
 * Map source file into memory as read-only.
 */
src_map = mmap(NULL,
                file_size,
                PROT_READ,
                MAP_PRIVATE,
                src_fd,
                0);

if (src_map == MAP_FAILED)
{
    perror("mmap source failed");
    close(src_fd);
    close(dest_fd);
    exit(EXIT_FAILURE);
}

/*
 * Map destination file into memory as read/write.
 */
dest_map = mmap(NULL,
                 file_size,
                 PROT_READ | PROT_WRITE,
                 MAP_SHARED,
```

```

printf("Time: %.9f seconds\n", traditional_time);

/*
 * Memory-mapped I/O
 */
start = get_time();

mmap_copy(source, mmap_output);

end = get_time();

mmap_time = end - start;

printf("\nMemory-mapped I/O completed.\n");
printf("Method: mmap() + memcpy() + msync()\n");
printf("Output: %s\n", mmap_output);
printf("Time: %.9f seconds\n", mmap_time);

/*
 * Comparison
 */
printf("\n===== \n");
printf("          PERFORMANCE COMPARISON\n");
printf("===== \n");

printf("Traditional I/O : %.9f seconds\n", traditional_time);
printf("mmap() I/O      : %.9f seconds\n", mmap_time);

printf("\nImplementation comparison:\n");
printf("Traditional I/O uses explicit read/write operations.\n");
printf("mmap() maps the file into memory and accesses it directly.\n");

printf("\nBoth methods completed successfully.\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ mkdir -p prac10task2
ruhi0404@Ruhitha:~/Operating_Systems$ cd prac10task2
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ pwd
/home/ruhi0404/Operating_Systems/prac10task2
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ nano input.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cat input.txt
Memory-mapped I/O allows a file to be mapped directly into
the process address space. This practical compares mmap()
with traditional read() and write() operations.
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ nano prac10task2.c
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ gcc -Wall -Wextra prac10ta
sk2.c -o prac10task2
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ ./prac10task2
=====
                MEMORY-MAPPED I/O PERFORMANCE TEST
=====

Source file: input.txt

Traditional I/O completed.
Method: read() + write()
Output: traditional_copy.txt
Time: 0.008813143 seconds

Memory-mapped I/O completed.
Method: mmap() + memcpy() + msync()
Output: mmap_copy.txt
Time: 0.010501862 seconds

=====
                PERFORMANCE COMPARISON
=====
Traditional I/O : 0.008813143 seconds
mmap() I/O      : 0.010501862 seconds

Implementation comparison:
Traditional I/O uses explicit read/write operations.
mmap() maps the file into memory and accesses it directly.

Both methods completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ ls -lh

```

```

ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ ls -lh
total 40K
-rw-r--r-- 1 ruhi0404 ruhi0404 165 Sep 28 14:43 input.txt
-rw-r--r-- 1 ruhi0404 ruhi0404 165 Sep 28 14:45 mmap_copy.txt
-rwxr-xr-x 1 ruhi0404 ruhi0404 17K Sep 28 14:45 prac10task2
-rw-r--r-- 1 ruhi0404 ruhi0404 5.4K Sep 28 14:45 prac10task2.c
-rw-r--r-- 1 ruhi0404 ruhi0404 165 Sep 28 14:45 traditional_copy.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cmp input.txt traditional_
copy.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cmp input.txt mmap_copy.tx
t
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cat mmap_copy.txt
Memory-mapped I/O allows a file to be mapped directly into
the process address space. This practical compares mmap()
with traditional read() and write() operations.
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cmp input.txt traditional_
copy.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ strace -f -e trace=openat,
read,write,mmap,munmap,msync,ftruncate,close ./prac10task2
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x73cee3226000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
mmap(NULL, 19875, PROT_READ, MAP_PRIVATE, 3, 0) = 0x73cee3221000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"
..., 832) = 832
mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x73cee300
0000
mmap(0x73cee3028000, 1609728, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_
_DENYWRITE, 3, 0x28000) = 0x73cee3028000
mmap(0x73cee31b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1b1000) = 0x73cee31b1000
mmap(0x73cee3200000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_
DENYWRITE, 3, 0x1ff000) = 0x73cee3200000
mmap(0x73cee3206000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_
ANONYMOUS, -1, 0) = 0x73cee3206000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x73cee321e000
munmap(0x73cee3221000, 19875) = 0

```

```

)
= 1
write(1, "===== "..., 45=====
=====
) = 45
write(1, "          PERFORMANCE COMPAR"..., 37          PERFORMANCE
COMPARISON
) = 37
write(1, "===== "..., 45=====
=====
) = 45
write(1, "Traditional I/O : 0.005325079 se"..., 38Traditional I/O : 0.005325
079 seconds
) = 38
write(1, "mmap() I/O      : 0.016889095 se"..., 38mmap() I/O      : 0.016889
095 seconds
) = 38
write(1, "\n", 1
)
= 1
write(1, "Implementation comparison:\n", 27Implementation comparison:
) = 27
write(1, "Traditional I/O uses explicit re"..., 53Traditional I/O uses expli
cit read/write operations.
) = 53
write(1, "mmap() maps the file into memory"..., 59mmap() maps the file into
memory and accesses it directly.
) = 59
write(1, "\n", 1
)
= 1
write(1, "Both methods completed successfu"..., 37Both methods completed suc
cessfully.
) = 37
+++ exited with 0 +++
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ cmp input.txt mmap_copy.tx
t
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ ls -l
input.txt
mmap_copy.txt
prac10task2
prac10task2.c
traditional_copy.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac10task2$ |

```

Observation:

The program successfully demonstrated file reading and writing using both traditional `read()` and `write()` system calls and memory-mapped I/O using `mmap()`. The same input file was copied using both approaches, and the generated files were verified to contain identical data using the `cmp` command. Execution time was measured for both methods to compare their efficiency. The traditional approach required explicit buffer-based read and write operations, while the memory-mapped approach mapped the file into the process address space and accessed the data directly through memory. The use of `mmap()`, `msync()`, and `munmap()` was also verified using `strace`. Thus, the practical successfully demonstrated the working, performance comparison, and implementation differences between traditional file I/O and memory-mapped I/O.